Combinatorial Privacy: Private Multi-Party Bitstream Grand Sum by Hiding in Birkhoff Polytopes

Praneeth Vepakomma
MBZUAI and MIT
vepakom@mit.edu

Abstract

We introduce *Combinatorial Privacy*, a paradigm where privacy derives from the non-uniqueness of decompositions in combinatorial structures, and first instantiate it with a naive PolyVeil protocol for private Boolean summation across clients. After demonstrating that naive variants succumb to a deterministic de-shuffling attack, we construct a two-layer protocol that achieves provable security. The main server receives only two aggregate sums and enjoys perfect simulation-based security against computationally unbounded adversaries. A separate aggregator receives matrices that encode private data via the Birkhoff polytope but does not see the random masks used to hide them; we prove that any likelihood-based attack against this aggregator requires solving #P-hard problems (permanent and mixed discriminant). Whether all polynomial-time attacks are ruled out remains an open conjecture. The protocol requires no public-key infrastructure, has $O(k)$ communication, and outputs exact results. This hybrid architecture — information-theoretic security for one entity, computational hardness from combinatorial structure for the other — distinguishes Combinatorial Privacy from traditional cryptographic (MPC, HE) and noise-based (DP) approaches.

Contents

1 Introduction **3**

2 Preliminaries **4**
2.1 Doubly Stochastic Matrices and the Birkhoff Polytope 4
2.2 Uniform Sampling of Permutation Matrices 5
2.3 Permutation Encoding of Binary Data 5
2.4 Algebraic Bit Count Extraction 5

3 Weak Protocol Variants and Their Vulnerabilities **7**
3.1 Problem Statement 7
3.2 Threat Model 7
3.3 Weak PolyVeil (Full Matrix Transmission) 8
3.4 Weak PolyVeil (Compressed) 9
3.5 Proof of Correctness (Weak Variants) 10
3.6 De-Shuffling Attack on the Weak Protocols 10
3.7 Summary of Vulnerabilities 11

4 The Two-Layer PolyVeil Protocol **11**
4.1 Architecture Overview . . . 11
4.2 Correctness . . . 12
4.3 Layer 1: Information-Theoretic Security of the Server . . . 12
4.4 Layer 2: Computational Security of the Aggregator via #P-Hardness . . . 12
4.4.1 The Aggregator's Inference Problem . . . 13
4.4.2 The Density of a Random Doubly Stochastic Matrix . . . 13
4.5 Formal Two-Layer Security Statement . . . 15
4.6 Why the Birkhoff Polytope Is Essential . . . 15
4.7 Geometry of the Protocol Parameter α^* . . . 16

5 Discussion **17**
5.1 Comparison with Generic Secure MPC . . . 17
5.2 Comparison with Differential Privacy . . . 18
5.3 Comparison with Homomorphic Encryption . . . 19
5.4 Communication Complexity . . . 19
5.5 Attack Resilience . . . 20
5.5.1 The Carlini et al. Attack on InstaHide and Why It Does Not Apply . . . 20
5.6 Post-Quantum Security . . . 20
5.7 Relationship to Ciphertext Indistinguishability . . . 20
5.8 Potential Attack Vectors . . . 21
5.9 Comparison with Alternative Approaches . . . 21

6 Conclusion **21**

A Worked Example (Weak Protocols) **23**
A.1 Ground Truth . . . 23
A.2 Client-Side Masking (Full Detail for Client 1) . . . 23
A.3 Client-Side Masking for Clients 2 and 3 (Summary) . . . 24
A.4 Server-Side Extraction and De-Shuffling . . . 24

A Background on Simulation-Based Security Proofs **24**
A.1 The Problem That Simulation Solves . . . 24
A.2 Formal Definition . . . 24
A.3 How to Construct a Simulation Proof . . . 25
A.4 Why Simulation Implies Security . . . 25
A.5 Application to PolyVeil . . . 25

1 Introduction

Computing aggregate statistics over private data held by many parties is a foundational problem in privacy-preserving computation. A concrete and widely applicable instance is the Boolean sum problem, in which k clients each hold a private binary string of length n and a server wishes to learn the total number of ones across all kn bits without learning any individual client's data. Applications range from epidemiological surveillance, where a health authority counts positive test results without accessing individual diagnoses, to federated analytics, where a service provider tallies binary feature occurrences without centralizing user data.

Existing approaches to this problem broadly fall into three categories. Generic secure multi-party computation (MPC) protocols, built on garbled circuits or secret sharing, provide strong composable security with guarantees based on specific computational hardness assumptions (such as the difficulty of integer factorization or the Learning with Errors problem over lattices), but impose communication and computation costs that scale with circuit complexity and become prohibitive when k is large or network conditions are constrained. Homomorphic encryption allows computation on ciphertexts, with security based on assumptions such as the composite residuosity problem (Paillier [5]) or the Learning with Errors problem (lattice-based FHE [6]), but carries substantial per-operation cost and requires careful key management. Differential privacy provides a framework for releasing aggregate statistics to untrusted parties by adding calibrated noise, offering formal (ε, δ)-guarantees that hold regardless of an adversary's computational power, but inherently sacrificing accuracy for privacy.

PolyVeil occupies a distinct point in this design space, one that we argue represents a new paradigm we call *Combinatorial Privacy*. In PolyVeil, the core idea is to encode each client's private bitstream as a permutation matrix, embed it inside a doubly stochastic matrix by mixing it with random decoy permutations, and use secure aggregation to recover the aggregate bit sum exactly. The Birkhoff–von Neumann theorem guarantees that every doubly stochastic matrix admits many decompositions into convex combinations of permutation matrices, and this non-uniqueness forms one of the two security layers of the protocol.

We first present two *weak* protocol variants (Algorithms 1 and 2) that are straightforward implementations of this idea. We then show that these variants are fundamentally insecure: the server can recover all individual data with probability 1 using a simple de-shuffling attack that exploits the integrality of the bit counts. Building on these weak protocols, we develop a corrected *two-layer protocol* (Algorithm 3) that achieves provable security. In the corrected protocol, the main server receives only aggregate scalars and is information-theoretically secure (its view is identically distributed for any two inputs with the same aggregate). A separate aggregator entity receives Birkhoff-encoded matrices but not the noise values, and faces a computational barrier: recovering the private permutation matrix from its encoding requires evaluating the density of a random doubly stochastic matrix at a given point, which we prove is #P-hard via a reduction to the permanent. The two layers compose so that no single entity can learn individual data: the server lacks the information, and the aggregator lacks the computational power.

This two-layer architecture distinguishes Combinatorial Privacy from existing paradigms. Unlike MPC and HE, which derive security from number-theoretic hardness (factoring, LWE), PolyVeil's computational layer derives from the #P-hardness of evaluating Birkhoff polytope decomposition likelihoods (proved for likelihood-based attacks; conjectured for all attacks). Unlike DP, PolyVeil produces exact answers. The protocol requires no public-key infrastructure and has $O(k)$ communication in the compressed variant.

Hardness assumptions and open problems. The security of the two-layer protocol rests on two pillars:

- The #P-hardness of computing the permanent of a 0/1 matrix (Valiant [14]) and the related #P-hardness of mixed discriminants (Barvinok [15]), which are classical results in complexity theory.
- The hardness of recovering a hidden permutation matrix from a convex combination with random permutations. We prove that *likelihood-based* attacks against the aggregator are #P-hard (Theorem 4.6). Whether *all* polynomial-time attacks are ruled out remains an open conjecture (Conjecture 4.5), which we pose as a challenge for future work.

The remainder of this paper is organized as follows. Section 2 establishes mathematical preliminaries. Section 3 presents the weak protocols and analyzes their vulnerabilities. Section 4 introduces the secure two-layer protocol and proves its security properties. Section 5 discusses comparisons with related work, communication costs, and attack resilience. Section 6 concludes. Appendix A provides background on simulation-based proofs.

2 Preliminaries

2.1 Doubly Stochastic Matrices and the Birkhoff Polytope

Definition 2.1 (Doubly stochastic matrix)**.** *A square matrix $A = (a_{ij}) \in \mathbb{R}^{m\times m}$ with non-negative entries is* doubly stochastic *if every row and every column sums to one: $\sum_{j=1}^{m} a_{ij} = 1$ for all $i \in [m]$, and $\sum_{i=1}^{m} a_{ij} = 1$ for all $j \in [m]$.*

Definition 2.2 (Birkhoff polytope)**.** *The* Birkhoff polytope *$\mathcal{B}_m$ is the set of all $m \times m$ doubly stochastic matrices. It is a convex polytope in $\mathbb{R}^{m\times m}$ of dimension $(m-1)^2$.*

Theorem 2.3 (Birkhoff–von Neumann [20, 21])**.** *The vertices of $\mathcal{B}_m$ are precisely the $m \times m$ permutation matrices. Every doubly stochastic matrix $A \in \mathcal{B}_m$ can be written as a convex combination of permutation matrices:*

$$A = \sum_{i=1}^{r} \theta_i P_i, \qquad \theta_i > 0, \quad \sum_{i=1}^{r} \theta_i = 1,$$

where each P_i is a permutation matrix. This is a Birkhoff–von Neumann (BvN) decomposition *of A.*

BvN decompositions are generically *non-unique*: a doubly stochastic matrix in the interior of $\mathcal{B}_m$ admits many distinct decompositions.

Theorem 2.4 (Decomposition multiplicity, Brualdi [17])**.** *Let $A \in \mathcal{B}_m$ have p positive entries. The number of distinct BvN decompositions of A is at least p^2.*

Theorem 2.5 (Marcus–Ree [18])**.** *Every $m \times m$ doubly stochastic matrix can be expressed as a convex combination of at most $m^2 - 2m + 2$ permutation matrices. Finding a BvN decomposition with the minimum number of permutations is NP-hard [19].*

2.2 Uniform Sampling of Permutation Matrices

A $m \times m$ permutation matrix P corresponds bijectively to a permutation $\sigma \in S_m$ (the symmetric group on $[m] = \{1, \ldots, m\}$) via $P_{ij} = \mathbf{1}[\sigma(i) = j]$. That is, row i of P has its unique 1 in column $\sigma(i)$. To draw P *uniformly at random from* S_m, one draws a uniformly random permutation σ and constructs the corresponding matrix.

The standard algorithm for drawing a uniform random permutation is the Fisher–Yates shuffle (also known as the Knuth shuffle): starting from the identity permutation $\sigma = (1, 2, \ldots, m)$, for $i = m, m-1, \ldots, 2$, draw j uniformly at random from $\{1, \ldots, i\}$ and swap $\sigma(i) \leftrightarrow \sigma(j)$. This produces each of the $m!$ permutations with equal probability $1/m!$ and runs in $O(m)$ time using $O(m)$ random bits (specifically $\sum_{i=2}^{m} \lceil \log_2 i \rceil$ bits). For our protocol with $m = 2n$, each random permutation matrix costs $O(n)$ time and $O(n \log n)$ random bits.

2.3 Permutation Encoding of Binary Data

Definition 2.6 (Bit-to-permutation encoding)**.** *For a bit $b \in \{0, 1\}$, define the 2×2 permutation matrix:*

$$\Pi(b) = \begin{cases} I_2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} & \text{if } b = 0, \\ J_2 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} & \text{if } b = 1. \end{cases}$$

For a bitstream $\mathbf{b} = (b_1, \ldots, b_n) \in \{0, 1\}^n$, define the block-diagonal encoding:

$$M(\mathbf{b}) = \operatorname{blockdiag}\big(\Pi(b_1),\ \Pi(b_2),\ \ldots,\ \Pi(b_n)\big) \in \{0, 1\}^{2n \times 2n}.$$

Since each $\Pi(b_j)$ is a 2×2 permutation matrix, their block-diagonal assembly $M(\mathbf{b})$ is a $2n \times 2n$ permutation matrix and a vertex of $\mathcal{B}_{2n}$. The block-diagonal structure confines each bit to a disjoint 2×2 block, enabling the algebraic extraction we now develop.

Example 2.7 (Encoding of a 2-bit stream)**.** *Consider the bitstream $\mathbf{b} = (1, 0)$ with $n = 2$. The encoding produces:*

$$M\big((1,0)\big) = \operatorname{blockdiag}(\Pi(1),\ \Pi(0)) = \operatorname{blockdiag}\left(\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}\right) = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

The first 2×2 block (rows 1–2, columns 1–2) encodes $b_1 = 1$ as the swap matrix J_2. The second 2×2 block (rows 3–4, columns 3–4) encodes $b_2 = 0$ as the identity I_2. All entries outside these blocks are zero due to the block-diagonal structure.

2.4 Algebraic Bit Count Extraction

Definition 2.8 (Selection vectors)**.** *Define $\mathbf{w}, \mathbf{y} \in \mathbb{R}^{2n}$ by:*

$$w_i = \begin{cases} 1 & \text{if } i \text{ is odd}, \\ 0 & \text{if } i \text{ is even}, \end{cases} \qquad y_i = \begin{cases} 0 & \text{if } i \text{ is odd}, \\ 1 & \text{if } i \text{ is even}. \end{cases}$$

Equivalently, $\mathbf{w} = (1, 0, 1, 0, \ldots, 1, 0)^T$ and $\mathbf{y} = (0, 1, 0, 1, \ldots, 0, 1)^T$, each of length $2n$.

Lemma 2.9 (Bit count extraction)**.** *For any bitstream* $\mathbf{b} = (b_1, \ldots, b_n) \in \{0,1\}^n$ *with permutation encoding* $M = M(\mathbf{b})$*:*

$$\mathbf{w}^T M\,\mathbf{y} \;=\; \sum_{j=1}^{n} b_j.$$

Proof. We compute $\mathbf{w}^T M\,\mathbf{y}$ by expanding the matrix-vector products step by step.

Computing $M\mathbf{y}$**.** Since $M = \mathrm{blockdiag}(\Pi(b_1), \ldots, \Pi(b_n))$ is block-diagonal, the product $M\mathbf{y}$ decomposes into independent block multiplications. For the j-th block, the relevant entries of $\mathbf{y}$ are $y_{2j-1} = 0$ and $y_{2j} = 1$. Writing $\mathbf{y}_j = (y_{2j-1},\, y_{2j})^T = (0,1)^T$ for the portion of $\mathbf{y}$ corresponding to block j, we have:

$$\Pi(b_j)\,\mathbf{y}_j = \Pi(b_j)\begin{pmatrix}0\\1\end{pmatrix}.$$

When $b_j = 0$: $\Pi(0)\binom{0}{1} = \begin{pmatrix}1 & 0\\0 & 1\end{pmatrix}\binom{0}{1} = \binom{0}{1}$. When $b_j = 1$: $\Pi(1)\binom{0}{1} = \begin{pmatrix}0 & 1\\1 & 0\end{pmatrix}\binom{0}{1} = \binom{1}{0}$. Therefore, the entries of $M\mathbf{y}$ at positions $2j-1$ and $2j$ are:

$$(M\mathbf{y})_{2j-1} = b_j, \qquad (M\mathbf{y})_{2j} = 1 - b_j. \tag{1}$$

To verify this: when $b_j = 0$, $(M\mathbf{y})_{2j-1} = 0 = b_j$ and $(M\mathbf{y})_{2j} = 1 = 1 - b_j$. When $b_j = 1$, $(M\mathbf{y})_{2j-1} = 1 = b_j$ and $(M\mathbf{y})_{2j} = 0 = 1 - b_j$. Both cases match (1).

Computing $\mathbf{w}^T(M\mathbf{y})$**.** Now we take the inner product of $\mathbf{w}$ with $M\mathbf{y}$:

$$\begin{aligned}
\mathbf{w}^T(M\mathbf{y}) &= \sum_{i=1}^{2n} w_i\,(M\mathbf{y})_i \\
&= \sum_{j=1}^{n}\Big[w_{2j-1}\,(M\mathbf{y})_{2j-1} \;+\; w_{2j}\,(M\mathbf{y})_{2j}\Big] \\
&= \sum_{j=1}^{n}\Big[1\cdot b_j \;+\; 0\cdot(1-b_j)\Big] \\
&= \sum_{j=1}^{n} b_j,
\end{aligned} \tag{2}$$

where the third equality substitutes $w_{2j-1} = 1$, $w_{2j} = 0$ from Definition 2.8 and $(M\mathbf{y})_{2j-1} = b_j$, $(M\mathbf{y})_{2j} = 1 - b_j$ from (1). The key mechanism is that $\mathbf{w}$ "selects" only the odd-indexed entries of $M\mathbf{y}$, each of which equals the corresponding bit b_j. □

Example 2.10 (Extraction for a concrete bitstream)**.** *Take* $n = 2$*,* $\mathbf{b} = (1,0)$*. Then* $\mathbf{w} = (1,0,1,0)^T$ *and* $\mathbf{y} = (0,1,0,1)^T$*. From Example 2.7:*

$$M = \begin{pmatrix}0 & 1 & 0 & 0\\1 & 0 & 0 & 0\\0 & 0 & 1 & 0\\0 & 0 & 0 & 1\end{pmatrix}.$$

First compute $M\mathbf{y}$*:*

$$M\mathbf{y} = \begin{pmatrix}0 & 1 & 0 & 0\\1 & 0 & 0 & 0\\0 & 0 & 1 & 0\\0 & 0 & 0 & 1\end{pmatrix}\begin{pmatrix}0\\1\\0\\1\end{pmatrix} = \begin{pmatrix}0\cdot0+1\cdot1+0\cdot0+0\cdot1\\1\cdot0+0\cdot1+0\cdot0+0\cdot1\\0\cdot0+0\cdot1+1\cdot0+0\cdot1\\0\cdot0+0\cdot1+0\cdot0+1\cdot1\end{pmatrix} = \begin{pmatrix}1\\0\\0\\1\end{pmatrix}.$$

Checking against (1)*: for block* $j = 1$ *(*$b_1 = 1$*):* $(M\mathbf{y})_1 = 1 = b_1$ *and* $(M\mathbf{y})_2 = 0 = 1 - b_1$*. For block* $j = 2$ *(*$b_2 = 0$*):* $(M\mathbf{y})_3 = 0 = b_2$ *and* $(M\mathbf{y})_4 = 1 = 1 - b_2$*. Both match.*

Now compute $\mathbf{w}^T(M\mathbf{y})$*:*

$$\mathbf{w}^T(M\mathbf{y}) = (1, 0, 1, 0) \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \end{pmatrix} = 1 \cdot 1 + 0 \cdot 0 + 1 \cdot 0 + 0 \cdot 1 = 1.$$

The result is $1 = b_1 + b_2 = 1 + 0$*, confirming the lemma.*

3 Weak Protocol Variants and Their Vulnerabilities

We first present two straightforward implementations of the core idea. These protocols are *weak* in that they are vulnerable to a simple de-shuffling attack that recovers all individual data with probability 1. They serve as stepping stones to the secure two-layer protocol.

3.1 Problem Statement

We consider k client entities, where client $t \in [k]$ holds a private binary bitstream $\mathbf{b}_t = (b_{t,1}, \ldots, b_{t,n}) \in \{0, 1\}^n$ of length n. Let $s_t = \sum_{j=1}^{n} b_{t,j}$ denote the number of ones in client t's bitstream. The goal is for a server to compute the aggregate $S = \sum_{t=1}^{k} s_t$ without learning any individual s_t, any individual bitstream $\mathbf{b}_t$, or any partial aggregates involving fewer than all k clients.

3.2 Threat Model

We operate in the *honest-but-curious* (semi-honest) model, defined as follows.

Server behavior. The server executes every instruction of the protocol exactly as specified. It does not deviate from the protocol by, for example, sending altered intermediate results to clients, injecting false data, or failing to perform a required computation. However, the server records every message it receives and may subsequently perform arbitrary polynomial-time computations on this recorded transcript in an attempt to infer individual client data. The server's computational power is bounded only by polynomial time: it may run brute-force searches over feasible spaces, solve optimization problems, and apply any statistical inference technique. The security guarantees we prove hold against any such polynomial-time analysis.

Communication security. All communication channels between each client and the server, and between each client and the shuffler, are authenticated and encrypted using standard transport-layer security (e.g., TLS 1.3). Authentication ensures that the server receives messages only from legitimate clients and not from impersonators. Encryption ensures that no external eavesdropper observing the network can read the content of any message. Together, these guarantees mean that the only entity that sees a client's message to the server is the server itself, and the only entity that sees a client's message to the shuffler is the shuffler itself. We do not assume that the communication channels hide metadata such as message timing or size.

Client behavior. Every client follows the protocol faithfully. No client modifies, omits, or fabricates any message. No client shares its private data, its random coins, or its intermediate computations with the server or with any other client (beyond what the protocol prescribes). In particular, no client colludes with the server to de-anonymize the shuffled values. The non-collusion assumption is essential: if even one client shared its η_t value directly with the server (outside the shuffle), the server could link that η_t to the client's identity and compute $s_t = (f_t - \eta_t)/\alpha^*$.

Trusted shuffler. There exists a functionality $\mathcal{F}_{\text{shuffle}}$ that operates as follows. It accepts as input one scalar value from each of the k clients, collecting the multiset $\{\eta_1, \ldots, \eta_k\}$. It then applies a permutation π drawn uniformly at random from the symmetric group S_k (the set of all $k!$ bijections on $[k]$), and outputs the permuted sequence $(\eta_{\pi(1)}, \ldots, \eta_{\pi(k)})$ to the server. Critically, the server learns the values in the output sequence but does *not* learn the permutation π. This means that for any position j in the output, the server knows the value $\eta_{\pi(j)}$ but cannot determine which client submitted it. The shuffler does not reveal π to the clients either.

This ideal functionality can be instantiated in several ways. The simplest is a *non-colluding auxiliary server*: a separate physical server, operated by an independent party that does not collude with the main server, receives all η_t values, permutes them, and forwards the result. A stronger instantiation is a *mixnet*, where each client encrypts its value under layered encryption addressed to a chain of relay servers, each of which peels one encryption layer and shuffles the messages. Verifiable mixnets [13] additionally produce a zero-knowledge proof that the output is a valid permutation of the input, preventing a malicious relay from altering values. A fully cryptographic instantiation uses a *multi-party shuffling protocol* such as the secret-shared shuffle of Chase, Ghosh, and Poburinnaya [13], which distributes the shuffling computation among two or more servers so that no single server learns the permutation, achieving security even if one server is corrupted.

We require $k \geq 3$ to ensure that the shuffle provides meaningful anonymity. With $k = 1$, the shuffled output contains a single value that is trivially linked to the sole client. With $k = 2$, the server has a $1/2$ probability of guessing the correct assignment, providing negligible privacy.

3.3 Weak PolyVeil (Full Matrix Transmission)

This variant sends the full doubly stochastic matrix to the server. It is vulnerable to the de-shuffling attack described later.

Algorithm 1 Weak PolyVeil

Public parameters: bit length n, scaling factor $\alpha^* \in (0,1)$, selection vectors $\mathbf{w}$, $\mathbf{y}$ (Definition 2.8).

<u>Client-Side Masking</u>

for each client $t \in \{1, \ldots, k\}$ in parallel **do**

  Encode private bitstream as $M_t = M(\mathbf{b}_t) \in \{0,1\}^{2n \times 2n}$ via Definition 2.6.

  Choose $K_t \geq 2$ and draw K_t uniformly random permutation matrices $P_{t,1}, \ldots, P_{t,K_t}$ from S_{2n} using the Fisher–Yates shuffle (Section 2.2).

  Sample positive coefficients $\alpha_{t,1}, \ldots, \alpha_{t,K_t} > 0$ with $\sum_{i=1}^{K_t} \alpha_{t,i} = 1 - \alpha^*$.

  Construct the masked matrix:

$$D_t \;=\; \alpha^* \, M_t \;+\; \sum_{i=1}^{K_t} \alpha_{t,i} \, P_{t,i}. \tag{3}$$

  Compute the noise term:

$$\eta_t \;=\; \sum_{i=1}^{K_t} \alpha_{t,i} \, \left(\mathbf{w}^T P_{t,i} \, \mathbf{y}\right). \tag{4}$$

  Send D_t to the server.

end for

<u>Server-Side Extraction</u>

for each client $t \in \{1, \ldots, k\}$ **do**

  Server computes:

$$f_t \;=\; \mathbf{w}^T D_t \, \mathbf{y}. \tag{5}$$

end for

<u>Secure Noise Transmission</u>

Each client t submits scalar η_t to the trusted shuffler $\mathcal{F}_{\text{shuffle}}$.

Shuffler draws $\pi \xleftarrow{\$} S_k$ and sends $(\eta_{\pi(1)}, \ldots, \eta_{\pi(k)})$ to server.

<u>Aggregation</u>

Server computes:

$$S \;=\; \frac{1}{\alpha^*} \left(\sum_{t=1}^{k} f_t \;-\; \sum_{t=1}^{k} \eta_t \right). \tag{6}$$

3.4 Weak PolyVeil (Compressed)

This variant avoids sending the matrix and instead sends only the extracted scalar f_t. It still suffers from the same de-shuffling attack.

Algorithm 2 Weak PolyVeil (Compressed)

Public parameter: $\alpha^* \in (0,1)$, selection vectors $\mathbf{w}$, $\mathbf{y}$.

for each client $t \in \{1, \ldots, k\}$ in parallel **do**

  Compute $s_t = \sum_{j=1}^{n} b_{t,j}$ locally.

  Draw random permutations $P_{t,1}, \ldots, P_{t,K_t}$ and coefficients $\alpha_{t,i} > 0$ with $\sum_i \alpha_{t,i} = 1 - \alpha^*$.

  Compute $\eta_t = \sum_i \alpha_{t,i} \left(\mathbf{w}^T P_{t,i}\, \mathbf{y}\right)$.

  Compute $f_t = \alpha^* s_t + \eta_t$.

  Send f_t to server.

end for

Clients jointly shuffle $\{\eta_1, \ldots, \eta_k\}$ and deliver to server.

Server computes $S = \frac{1}{\alpha^*}\left(\sum_t f_t - \sum_t \eta_t\right)$.

The compressed variant offers three concrete advantages beyond the communication reduction from $O(n^2)$ to $O(1)$ per client. First, the server never sees the doubly stochastic matrix D_t, which eliminates the BvN decomposition as an attack vector: the server has no matrix to decompose. Second, client-side computation drops from $O(n^2)$ (constructing a $2n \times 2n$ matrix) to $O(nK_t)$ (generating K_t random permutations and computing K_t bilinear forms). Third, server computation drops from $O(kn^2)$ to $O(k)$, becoming independent of the bitstream length.

The trade-off is that a malicious client can send an arbitrary f_t without the server being able to verify it, since the server no longer has D_t to check. Under the semi-honest model this is not a concern.

3.5 Proof of Correctness (Weak Variants)

Theorem 3.1 (Correctness of weak protocols)**.** *Algorithms 1 and 2 compute $S = \sum_{t=1}^{k} s_t$ exactly.*

Proof. The matrix M_t is a permutation matrix (Definition 2.6) and hence doubly stochastic. Each $P_{t,i}$ is a permutation matrix drawn from S_{2n} and is likewise doubly stochastic. The coefficients $\alpha^*, \alpha_{t,1}, \ldots, \alpha_{t,K_t}$ are strictly positive with $\alpha^* + \sum_i \alpha_{t,i} = 1$. Since $\mathcal{B}_{2n}$ is convex, $D_t = \alpha^* M_t + \sum_i \alpha_{t,i} P_{t,i} \in \mathcal{B}_{2n}$.

Applying the bilinear form $A \mapsto \mathbf{w}^T A\, \mathbf{y}$ to D_t and using linearity:

$$\begin{aligned} f_t = \mathbf{w}^T D_t\, \mathbf{y} &= \alpha^* \left(\mathbf{w}^T M_t\, \mathbf{y}\right) + \sum_{i=1}^{K_t} \alpha_{t,i} \left(\mathbf{w}^T P_{t,i}\, \mathbf{y}\right) \\ &= \alpha^*\, s_t + \eta_t, \end{aligned} \tag{7}$$

where we used Lemma 2.9 for $\mathbf{w}^T M_t \mathbf{y} = s_t$ and definition (4) for η_t.

Summing over all k clients: $\sum_{t=1}^{k} f_t = \alpha^* \sum_{t=1}^{k} s_t + \sum_{t=1}^{k} \eta_t$. The server knows $\sum_t f_t$ from the extraction step and $\sum_t \eta_t$ from the shuffled list (the sum is invariant under permutation). Rearranging gives $\sum_t s_t = \frac{1}{\alpha^*}(\sum_t f_t - \sum_t \eta_t)$, which is exactly (6). □

3.6 De-Shuffling Attack on the Weak Protocols

The weak protocols are fatally insecure. The server receives identity-linked f_t values and the shuffled η_t values. Because s_t is an integer between 0 and n, the server can uniquely identify the correct assignment of η_t to clients.

Theorem 3.2 (De-shuffling via integrality constraint)**.** *In both weak protocols, the server can recover every client's bit count s_t with probability 1 (over the protocol's randomness).*

Proof. The server's strategy is as follows. For each candidate assignment $\sigma\colon [k] \to [k]$ (a bijection from client indices to shuffled positions), compute $\hat{s}_t(\sigma) = (f_t - \tilde{\eta}_{\sigma(t)})/\alpha^*$ for all $t \in [k]$.

For the true assignment $\sigma^* = \pi^{-1}$:

$$\hat{s}_t(\sigma^*) = \frac{f_t - \eta_t}{\alpha^*} = \frac{\alpha^* s_t + \eta_t - \eta_t}{\alpha^*} = s_t \ \in \ \{0, 1, \ldots, n\}.$$

For any wrong assignment $\sigma \neq \sigma^*$, there exists at least one t with $\sigma(t) \neq \sigma^*(t)$, i.e., $\tilde{\eta}_{\sigma(t)} = \eta_j$ for some $j \neq t$. Then:

$$\hat{s}_t(\sigma) = s_t + \frac{\eta_t - \eta_j}{\alpha^*}.$$

This is an integer if and only if $(\eta_t - \eta_j)/\alpha^*$ is an integer. Since $\eta_t = \sum_{i=1}^{K_t} \alpha_{t,i}(\mathbf{w}^T P_{t,i}\mathbf{y})$ is a linear combination of integers with *continuously distributed* random coefficients $\alpha_{t,i}$, the difference $\eta_t - \eta_j$ is a continuous random variable. The probability that a continuous random variable equals any specific value (in particular, any integer multiple of α^*) is zero.

Therefore, with probability 1 over the protocol's randomness, the true assignment σ^* is the *unique* assignment for which all k candidate values are non-negative integers at most n. The server identifies σ^* and recovers all s_t values exactly. □

Remark 3.3 (Worked example)**.** *The worked example in Appendix A (originally Section* **??***) illustrates this attack explicitly. The server tests all* $3! = 6$ *assignments of shuffled* η *values to clients and finds that exactly one assignment yields valid bit counts* $(s_1, s_2, s_3) = (1, 2, 1)$ *with all* $s_t \in \{0, 1, 2\}$.

3.7 Summary of Vulnerabilities

Both weak protocols reveal all individual s_t to the server. The root cause is that the server receives identity-linked f_t values *and* the shuffled η_t values *separately*, and the integrality of s_t provides a verification criterion that uniquely identifies the true shuffle permutation. This vulnerability motivates the secure two-layer protocol presented next.

4 The Two-Layer PolyVeil Protocol

We now present a corrected protocol with a *two-layer* architecture in which no single entity can learn individual data. The design separates information so that the server receives only aggregate scalars, while the aggregator receives Birkhoff-encoded matrices but not the noise values. The server's security is information-theoretic; the aggregator's security is computational, based on the hardness of recovering a permutation from a convex combination.

4.1 Architecture Overview

The protocol involves three types of entities:

- **Clients** $t \in [k]$: each holds private $\mathbf{b}_t \in \{0, 1\}^n$.
- **Aggregator** $\mathcal{A}$: receives masked matrices D_t and computes the scalar aggregate $F = \sum_t \mathbf{w}^T D_t \mathbf{y}$, which it sends to the server. The aggregator does *not* receive η_t values.
- **Server** $\mathcal{S}$: receives F from the aggregator and $H = \sum_t \eta_t$ from a separate noise-aggregation channel, and computes $S = (F - H)/\alpha^*$.

The key design principle is *separation of information*: the aggregator sees D_t (which encodes M_t) but not η_t; a separate channel delivers $\sum \eta_t$ to the server without the aggregator's involvement.

Algorithm 3 Two-Layer PolyVeil

Public parameters: n, $\alpha^* \in (0,1)$, $\mathbf{w}$, $\mathbf{y}$.
Entities: Aggregator $\mathcal{A}$, noise aggregator $\mathcal{B}$, server $\mathcal{S}$.

Client-Side Computation
for each client $t \in [k]$ in parallel **do**
  Encode $M_t = M(\mathbf{b}_t)$. Draw $K_t \geq 2$ uniform random permutations $P_{t,i} \in S_{2n}$.
  Sample $\alpha_{t,1}, \ldots, \alpha_{t,K_t} > 0$ with $\sum_i \alpha_{t,i} = 1 - \alpha^*$.
  Compute $D_t = \alpha^* M_t + \sum_i \alpha_{t,i} P_{t,i}$ and $\eta_t = \sum_i \alpha_{t,i}(\mathbf{w}^T P_{t,i}\mathbf{y})$.
  Send D_t to aggregator $\mathcal{A}$. Send η_t to noise aggregator $\mathcal{B}$.
end for
Aggregation
$\mathcal{A}$ computes $F = \sum_{t=1}^{k} \mathbf{w}^T D_t \mathbf{y}$ and sends F to server $\mathcal{S}$.
$\mathcal{B}$ computes $H = \sum_{t=1}^{k} \eta_t$ and sends H to server $\mathcal{S}$.
$\mathcal{S}$ computes $S = (F - H)/\alpha^*$.

4.2 Correctness

Theorem 4.1 (Correctness)**.** *Algorithm 3 outputs $S = \sum_t s_t$ exactly.*

Proof. $F = \sum_t \mathbf{w}^T D_t \mathbf{y} = \sum_t (\alpha^* s_t + \eta_t) = \alpha^* S + H$. Therefore $(F - H)/\alpha^* = S$. □

4.3 Layer 1: Information-Theoretic Security of the Server

The server $\mathcal{S}$ receives only two scalars (F, H).

Theorem 4.2 (Perfect simulation-based security of the server)**.** *For any two input configurations $\mathcal{D}$ and $\mathcal{D}'$ with $\sum_t s_t = \sum_t s'_t = S$, the server's view (F, H) is identically distributed. There exists a simulator $\mathcal{S}^{\text{sim}}$ such that $\mathcal{S}^{\text{sim}}(S) \equiv (F, H)$ for all inputs with aggregate S.*

Proof. Each η_t depends only on decoy randomness (permutations and coefficients), not on $\mathbf{b}_t$. Let μ denote the distribution of a single η_t. Then $H = \sum_t \eta_t \sim \mu^{*k}$ has a distribution independent of $\mathcal{D}$. Since $F = \alpha^* S + H$ is a deterministic function of S and H, the view $(F, H) = (\alpha^* S + H,\ H)$ depends on $\mathcal{D}$ only through S.

The simulator draws $\hat{H} \sim \mu^{*k}$ by generating fresh decoy randomness and outputs $(\alpha^* S + \hat{H},\ \hat{H})$. This is identically distributed to the real view. □

This is information-theoretic security: statistical distance zero, holding against computationally unbounded adversaries. The proof uses only that η_t is independent of $\mathbf{b}_t$ — any noise generation process with this property would suffice for the server's security alone.

4.4 Layer 2: Computational Security of the Aggregator via #P-Hardness

The aggregator $\mathcal{A}$ sees the individual matrices $D_1, \ldots, D_k$ but does *not* receive η_t or f_t as separate scalars. To recover client t's data, the aggregator must extract M_t from $D_t = \alpha^* M_t + (1 - \alpha^*) R_t$. We now show that this task is computationally hard.

4.4.1 The Aggregator's Inference Problem

Given D_t, the aggregator can enumerate the set of *consistent* permutation matrices:

$$\mathcal{L}(D_t, \alpha^*) = \{M' \in S_{2n} : (D_t - \alpha^* M')/(1-\alpha^*) \in \mathcal{B}_{2n}\}.$$

Under the interior condition ($\alpha^* \leq \min_{ij}(D_t)_{ij}$, Proposition 4.11), this set contains all $(2n)!$ permutation matrices. But consistency alone does not prevent inference: the aggregator can compute the *posterior probability* of each candidate:

$$\Pr(M_t = M' \mid D_t) \;=\; \frac{\nu(R')}{\sum_{M'' \in \mathcal{L}} \nu(R'')}, \tag{8}$$

where $R' = (D_t - \alpha^* M')/(1-\alpha^*)$ and ν is the density of the random decoy component R_t over $\mathcal{B}_{2n}$.

Computing this posterior requires evaluating the density $\nu(R')$. We now explain why this evaluation is #P-hard.

4.4.2 The Density of a Random Doubly Stochastic Matrix

Definition 4.3 (Random doubly stochastic matrix from random convex combination)**.** *Let $P_1, \ldots, P_K$ be i.i.d. uniform random permutation matrices in S_{2n}, and let $(\alpha_1, \ldots, \alpha_K)$ be positive coefficients with $\sum_i \alpha_i = 1 - \alpha^*$, drawn from some continuous distribution on the simplex. The random doubly stochastic matrix $R = \frac{1}{1-\alpha^*} \sum_i \alpha_i P_i$ has a distribution ν over $\mathcal{B}_{2n}$.*

The density $\nu(R')$ at a point $R' \in \mathcal{B}_{2n}$ is the probability (density) that the random process produces exactly R'. Computing this requires marginalizing over all possible permutation matrices and coefficient vectors that yield R':

$$\nu(R') = \sum_{\sigma_1, \ldots, \sigma_K \in S_{2n}} \int_{\Delta_K} \mathbf{1}\left[\frac{1}{1-\alpha^*} \sum_{i=1}^{K} \alpha_i P_{\sigma_i} = R'\right] \cdot \frac{1}{((2n)!)^K} \cdot g(\alpha_1, \ldots, \alpha_K)\, d\alpha, \tag{9}$$

where g is the density of the coefficient distribution on the K-simplex, and $\Delta_K = \{(\alpha_1, \ldots, \alpha_K) : \alpha_i > 0,\ \sum \alpha_i = 1 - \alpha^*\}$.

Why this is hard: The indicator in the integral forces each P_{σ_i} to lie in the support of R', i.e., every P_{σ_i} must have ones only where R' is positive. The number of permutations consistent with this support is the permanent of the 0/1 matrix $A(R')$ where $A_{ij} = \mathbf{1}[R'_{ij} > 0]$. By Valiant's theorem, computing this permanent is #P-complete. The summation thus involves a #P-hard number of terms, and the integral further involves mixed volumes (or mixed discriminants) that are also #P-hard to compute (Barvinok [15]). Therefore, evaluating $\nu(R')$ exactly (or even approximating it sufficiently well) is #P-hard.

We make this precise with the following proposition and theorem.

Proposition 4.4 (Evaluating ν requires counting perfect matchings)**.** *The summation in* (9) *ranges over all K-tuples of permutation matrices in* $\mathrm{Supp}(R') = \{Q \in S_{2n} : Q_{ij} = 1 \implies R'_{ij} > 0\}$*. The cardinality* $|\mathrm{Supp}(R')| = \mathrm{perm}(A(R'))$*, where $A_{ij} = \mathbf{1}[R'_{ij} > 0]$ and* perm *denotes the permanent. By Valiant [14], computing* $\mathrm{perm}(A)$ *for 0/1 matrices is #P-complete. The summation in* (9) *therefore has* $\mathrm{perm}(A(R'))^K$ *nonzero terms.*

Proof. For the indicator in (9) to be nonzero, every P_{σ_i} must have $P_{\sigma_i,ab} = 1$ only at positions where $R'_{ab} > 0$ (since $\alpha_i > 0$). This means $P_{\sigma_i} \in \mathrm{Supp}(R')$. The cardinality of $\mathrm{Supp}(R')$ equals the number of perfect matchings in the bipartite graph on positions (a, b) with $R'_{ab} > 0$, which is $\mathrm{perm}(A(R'))$. □

Proposition 4.4 establishes that the *natural algorithm* for computing the posterior — evaluate $\nu(R')$ for each candidate M' using (9) — requires summing over a #P-hard number of terms. However, this does not rule out the existence of alternative algorithms that recover M_t without computing ν.

Conjecture 4.5 (Computational hardness of the aggregator's inference)**.** *No polynomial-time algorithm can, given D_t drawn from $\alpha^* M_t + (1-\alpha^*)\nu$ (where M_t is a uniformly random permutation matrix), recover M_t with probability non-negligibly better than $1/(2n)!$ (the success probability of random guessing).*

We do not prove this conjecture in full generality. A complete proof would require showing that *any* efficient algorithm for recovering M_t can be transformed into an efficient algorithm for a #P-hard problem — a reduction in the opposite direction from what Proposition 4.4 establishes. Such a reduction faces two technical obstacles: (a) the permanent is #P-hard in the *worst case*, whereas the aggregator faces a *distributional* instance (the residual R was drawn from ν), and average-case hardness of the permanent is an open problem [16]; and (b) even worst-case hardness of evaluating ν does not rule out algorithms that bypass density evaluation entirely.

We do, however, prove the following supporting result.

Theorem 4.6 (Aggregator security against likelihood-based attacks)**.** *Any algorithm that recovers M_t from D_t by computing or approximating the likelihood ratio $\nu(R')/\nu(R'')$ for candidate permutation matrices M', M'' must solve an instance of the* mixed discriminant *problem, which is #P-hard (Barvinok, 1996 [15]).*

Proof. The likelihood ratio for two candidates M' and M'' is $\nu(R')/\nu(R'')$ where $R' = (D_t - \alpha^* M')/(1-\alpha^*)$ and $R'' = (D_t - \alpha^* M'')/(1-\alpha^*)$.

Using (9), each $\nu(R)$ is a sum over K-tuples of permutations in $\mathrm{Supp}(R)$, weighted by the integral of the coefficient density g over the simplex face defined by the linear constraint $\sum_i \alpha_i P_{\sigma_i} = (1-\alpha^*)R$.

For each fixed K-tuple $(\sigma_1, \ldots, \sigma_K)$, the constraint $\sum_i \alpha_i P_{\sigma_i} = (1-\alpha^*)R$ defines a polytope in $\mathbb{R}^K$ (the intersection of the simplex with a linear subspace), and the integral of g over this polytope is a mixed volume computation. Computing mixed volumes of polytopes defined by 0/1 matrices is equivalent to computing the *mixed discriminant* of a collection of positive semidefinite matrices, which Barvinok [15] showed is #P-hard.

Concretely, for $K = 2n$ and uniform coefficient distribution ($g = \mathrm{Dirichlet}(1, \ldots, 1)$ density), the integral in each term of (9) equals the volume of a polytope defined by the permutation matrices, and the total sum over all valid tuples is a mixed discriminant. Since computing this sum is #P-hard, any algorithm that computes or approximates the likelihood ratio must solve a #P-hard problem. □

Remark 4.7 (Significance and limitations)**.** *Theorem 4.6 proves that the* most natural class of attacks *— those based on Bayesian posterior computation — is computationally intractable. This eliminates the MAP estimator and any algorithm that ranks candidates by likelihood. It does* not *rule out algorithms that exploit structural properties of D_t (e.g., spectral methods, linear programming relaxations) to recover M_t without computing likelihoods.*

We contrast this with the situation for other noise distributions. If the decoy component were additive Gaussian noise ($D_t = \alpha^* M_t + \varepsilon$, $\varepsilon \sim N(0, \sigma^2 I)$), the likelihood $\Pr(D_t \mid M')$ would be a simple Gaussian density, computable in $O(n^2)$ time. The MAP estimate would equal the nearest permutation matrix to D_t/α^* in Frobenius norm, solvable by the Hungarian algorithm in $O(n^3)$. The Birkhoff encoding is therefore strictly harder to invert than Gaussian noise, because the likelihood involves a #P-hard summation that Gaussian noise avoids.

4.5 Formal Two-Layer Security Statement

Definition 4.8 (Two-layer security)**.** *A protocol has* two-layer security *against semi-honest adversaries if:*

(i) ***Information-theoretic server security:*** *the server's view is identically distributed for any two inputs with the same aggregate (statistical distance zero).*

(ii) ***Computational aggregator security against likelihood attacks:*** *no polynomial-time aggregator can compute or approximate the posterior distribution* (8) *over candidate permutation matrices, because doing so requires solving a #P-hard problem.*

(iii) ***Non-collusion:*** *the server and aggregator do not share their views.*

Theorem 4.9 (Two-Layer PolyVeil security — proved components)**.** *Under the non-collusion assumption, Algorithm 3 achieves:*

- *condition (i) by Theorem 4.2 (perfect simulation, unconditional);*
- *condition (ii) by Theorem 4.6 (likelihood-based attacks require solving #P-hard mixed discriminant), under the interior condition of Proposition 4.11.*

Remark 4.10 (What remains open)**.** *Full aggregator security (Conjecture 4.5) — ruling out* all *polynomial-time attacks, not just likelihood-based ones — remains an open problem. A proof would require either (a) a reduction from a #P-complete or NP-hard problem to the search problem "recover M_t from D_t," or (b) an average-case hardness result for the permanent over the specific distribution induced by ν. Both are significant open problems in computational complexity. The average-case hardness of the permanent has been studied by Lipton [16] and others, with partial results (e.g., hardness over finite fields) but no complete resolution over the reals.*

4.6 Why the Birkhoff Polytope Is Essential

We now explain precisely where the Birkhoff polytope machinery contributes and where it does not.

What the Birkhoff polytope does NOT provide. The server's information-theoretic security (Theorem 4.2) does not depend on the Birkhoff polytope. Any method of generating noise η_t independently of $\mathbf{b}_t$ would achieve the same result. In particular, the trivial protocol "each client sends s_t + noise to a trusted aggregator" achieves the same server security. This observation, correctly made in [4] and subsequent work on secure aggregation, is not the contribution of this paper.

What the Birkhoff polytope DOES provide. The aggregator's computational security against likelihood-based attacks (Theorem 4.6) *requires* the Birkhoff polytope. The aggregator sees $D_t = \alpha^* M_t + (1-\alpha^*)R_t$ and must extract M_t. The difficulty of this extraction is a consequence of three properties of the Birkhoff polytope, all of which are necessary:

1. **Super-exponentially many vertices.** $\mathcal{B}_{2n}$ has $(2n)!$ vertices (permutation matrices). This means the number of candidate explanations grows faster than $2^{cn\log n}$, providing a large search space.

2. **Non-uniqueness of decomposition.** Every interior point of $\mathcal{B}_{2n}$ admits multiple BvN decompositions. Under the interior condition, every permutation matrix is a *valid* component of some decomposition of D_t. This means the set of consistent candidates is as large as possible.

3. **#P-hardness of the likelihood.** Computing the posterior probability of each candidate M' requires evaluating $\nu(R')$, which involves a summation over $\mathrm{perm}(A(R'))^K$ terms (Proposition 4.4) and mixed discriminant computations that are #P-hard (Theorem 4.6). This is the crucial ingredient: it is not enough that many decompositions *exist* — the *likelihood-based approach* to determining which one was used must be computationally intractable. Whether non-likelihood-based approaches are also hard remains an open problem (Conjecture 4.5).

These three properties together constitute what we call *Combinatorial Privacy*: privacy derived from the hardness of inverting a combinatorial embedding. The paradigm is distinct from noise-based privacy (DP), hardness-based encryption (MPC, HE using factoring/LWE), and secret-sharing-based protocols.

4.7 Geometry of the Protocol Parameter α^*

The Birkhoff polytope $\mathcal{B}_{2n}$ is a convex body whose vertices are the $(2n)!$ permutation matrices. Its *boundary* consists of doubly stochastic matrices with at least one zero entry. Its *interior* $\mathrm{int}(\mathcal{B}_{2n})$ consists of matrices with all entries strictly positive. A matrix "deep in the interior" has its minimum entry bounded away from zero, meaning it is geometrically far from every face.

The masked matrix $D_t = \alpha^* M_t + (1-\alpha^*)R_t$ lies on the line segment in $\mathcal{B}_{2n}$ from the random interior point $R_t = \frac{1}{1-\alpha^*}\sum_i \alpha_{t,i}P_{t,i}$ to the vertex M_t, at parameter α^* of the way from R_t toward M_t.

Large α^* (close to 1). $D_t \approx M_t$ lies near a vertex. Most entries are close to zero, revealing the structure of M_t. The number of consistent alternative permutations $L(D_t, \alpha^*)$ is small because $(D_t)_{ij} < \alpha^*$ at many positions.

Small α^* (close to 0). $D_t \approx R_t$ lies in the interior with all entries positive. The condition $(D_t)_{ij} \geq \alpha^*$ holds everywhere, giving $L(D_t, \alpha^*) = (2n)!$.

The precise condition. For every permutation matrix M' to be consistent with D_t, we need $(D_t)_{ij} \geq \alpha^*$ wherever $M'_{ij} = 1$. Since every position (i,j) has $M'_{ij} = 1$ for some permutation, the condition is $\alpha^* \leq \min_{ij}(D_t)_{ij}$. Since $\min_{ij}(D_t)_{ij} \geq (1-\alpha^*)\min_{ij}(R_t)_{ij}$, a sufficient condition is $\alpha^* \leq \frac{\min_{ij}(R_t)_{ij}}{1+\min_{ij}(R_t)_{ij}}$.

Proposition 4.11. *The condition* $\alpha^* \leq \min_{ij}(D_t)_{ij}$ *holds with probability at least* $1-\delta$ *when* $K_t \geq 2n\ln(4n^2/\delta)$ *and* $\alpha^* \leq \min_l \alpha_{t,l}$.

Proof. Entry (i,j) of R_t is zero iff all K_t decoys have 0 there. Each has $\Pr[P_{ij}=1] = 1/(2n)$, so $\Pr[(R_t)_{ij}=0] = (1-1/(2n))^{K_t}$. Union bound over $(2n)^2$ entries gives $\Pr[\min_{ij}(R_t)_{ij}=0] \leq 4n^2 e^{-K_t/(2n)} \leq \delta$ when $K_t \geq 2n\ln(4n^2/\delta)$. Conditional on all entries positive, $\min_{ij}(D_t)_{ij} \geq \min_l \alpha_{t,l}$, so $\alpha^* \leq \min_l \alpha_{t,l}$ suffices. □

Practical guidance. For $n = 1000$, setting $K_t \approx 30{,}000$ and $\alpha^* = 1/(4n) = 0.00025$ ensures the condition with probability exceeding $1-10^{-6}$. Very small α^* (e.g., 10^{-15}) causes floating-point errors in the subtraction $\sum f_t - \sum \eta_t$. A practical range is $\alpha^* \in [1/(4n), 1/(2n)]$.

5 Discussion

We now discuss how the two-layer protocol compares with existing approaches, its communication efficiency, attack resilience, and other practical considerations.

5.1 Comparison with Generic Secure MPC

Circuit complexity of the Boolean sum. To understand the concrete cost of MPC for this problem, we must first determine the circuit that computes the Boolean sum $S = \sum_{t=1}^{k} s_t = \sum_{t=1}^{k}\sum_{j=1}^{n} b_{t,j}$. This is a sum of $N = kn$ bits, producing an output in $\{0,1,\ldots,N\}$ representable in $\lceil \log_2(N+1) \rceil$ bits. An addition tree with $N-1$ adder stages, each adding two $\lceil \log_2 N \rceil$-bit numbers, requires $O(N \log N) = O(kn\log(kn))$ Boolean gates. This is the circuit size $|C|$ for this problem.

In garbled-circuit-based MPC (e.g., Yao's protocol for two parties, or its multi-party extensions), each Boolean gate requires one or more oblivious transfers (OTs), each costing $O(\kappa)$ bits of communication where κ is the security parameter (typically 128). The total communication is therefore $O(|C| \cdot \kappa) = O(kn\log(kn) \cdot \kappa)$ per pair of parties, and in a k-party setting the naive approach requires $O(k^2)$ pairwise executions, giving total communication $O(k^2 n \log(kn) \cdot \kappa)$. Even with OT extension techniques that amortize the per-OT cost, the circuit-proportional overhead remains.

In secret-sharing-based MPC (e.g., SPDZ or its variants), each addition gate is "free" (requiring only local computation) but each multiplication gate requires a preprocessed Beaver triple and $O(k\kappa)$ bits of communication. For the Boolean sum, the circuit consists entirely of additions if the inputs are in an arithmetic field, making the online phase very cheap at $O(k)$ communication. However, the offline phase for generating Beaver triples (needed for the multiplication gates in the binary-to-arithmetic conversion) and the input-sharing phase still require $O(kn\kappa)$ communication. Furthermore, secret-sharing MPC requires an interactive setup round where each client secret-shares its input among all parties, costing $O(k^2 n)$ communication.

PolyVeil's advantage. In PolyVeil's compressed variant, each client sends a single scalar f_t to the server and a single scalar η_t to the shuffler. The total communication is $O(k)$ real numbers, independent of n, κ, and any circuit complexity. For a concrete comparison, consider $k = 1000$ clients with $n = 10{,}000$ bits each at security parameter $\kappa = 128$. The garbled-circuit MPC approach would require on the order of $k^2 n \log(kn)\kappa \approx 10^6 \times 10^4 \times 24 \times 128 \approx 3 \times 10^{13}$ bits ≈ 3.8 terabytes of total communication. Secret-sharing approaches reduce this substantially but still require $O(kn\kappa) \approx$

$10^3 \times 10^4 \times 128 \approx 1.3 \times 10^9$ bits $\approx$ 160 megabytes for input sharing alone. PolyVeil's compressed variant requires $2k = 2000$ scalars $\approx$ 16 kilobytes. The gap is several orders of magnitude.

PolyVeil is not MPC. Despite the superficial similarity (multiple parties compute a joint function while preserving privacy), PolyVeil is not an MPC protocol in the standard sense. MPC protocols achieve security through cryptographic operations applied to the computation itself (garbling gates, sharing secrets, generating correlated randomness), and they can compute *any* function. PolyVeil achieves security through a different mechanism entirely — the combinatorial structure of the Birkhoff polytope — and is limited to summation (and linear functions more broadly). It does not garble circuits, share secrets among parties, or perform homomorphic operations. The only cryptographic component is the shuffler, which is a *communication primitive* rather than a computational one.

A more precise characterization is that PolyVeil is an instance of the Combinatorial Privacy paradigm, in which privacy derives from embedding private data in a combinatorial object with non-unique decompositions. It belongs to the broader family of protocols that use structured noise and anonymous communication to achieve privacy, alongside the shuffle model of differential privacy and mixnet-based anonymous messaging. The novelty of PolyVeil within this family is the use of the Birkhoff polytope's geometry to generate the structured noise.

When to prefer MPC over PolyVeil. MPC is the right choice when the target function is more complex than summation (e.g., computing a median, training a model, or evaluating a decision tree), when security against malicious adversaries is required, or when the protocol must compose securely with other cryptographic primitives in a larger system (UC security). For the specific problem of private Boolean sums in the semi-honest model, PolyVeil is more efficient by orders of magnitude.

5.2 Comparison with Differential Privacy

The fundamental difference is accuracy. In the local DP model, where each client randomizes their data independently, the error for estimating a Boolean sum over kn bits scales as $\Theta(\sqrt{kn}/\varepsilon)$, which grows with the number of clients. In the central model, where a trusted curator adds noise to the true aggregate, the error is $\Theta(n/\varepsilon)$, independent of k but still nonzero. The shuffle model interpolates, achieving central-model accuracy with local-model trust assumptions. PolyVeil produces exact results with zero error. When an application requires the precise count (e.g., regulatory reporting, election tallying, financial reconciliation), no DP mechanism can substitute for PolyVeil.

The trust assumptions of PolyVeil and shuffle-model DP are essentially identical, as both require a shuffler. The difference is where the privacy comes from. In shuffle-model DP, privacy arises from statistical noise combined with shuffling anonymity. In PolyVeil, privacy arises from algebraic masking (the BvN non-uniqueness) combined with shuffling anonymity. PolyVeil's approach is deterministic and exact, while shuffle-model DP is inherently randomized and approximate.

A further advantage of PolyVeil over central DP is that the server never observes individual client data, not even before noise addition. In central DP, the curator sees all raw data and must be trusted to add noise correctly. A compromised or dishonest curator can simply omit the noise and read the data in the clear. In PolyVeil, the server sees only masked doubly stochastic matrices (full protocol) or scalar f_t values (compressed protocol), and no individual bitstream is ever exposed to any party other than the client that holds it.

PolyVeil is forward-compatible with DP. After computing the exact aggregate S via PolyVeil, the server can add discrete Gaussian noise $Z \sim \mathcal{N}_{\mathbb{Z}}(0, \sigma^2)$ with $\sigma \geq n\sqrt{2\ln(1.25/\delta)}/\varepsilon$ to produce a privatized output $\tilde{S} = S + Z$ satisfying (ε, δ)-DP. The sensitivity of S with respect to replacing one client's bitstream is n (a client can change from all zeros to all ones), giving the standard Gaussian mechanism calibration. This composition provides two layers of protection: PolyVeil's cryptographic guarantee protects individual data during computation, and the DP guarantee protects the aggregate at release. The two layers compose cleanly because, by the simulation proof, the server's intermediate view is a deterministic function of S (in distribution), and adding noise to S is post-processing.

5.3 Comparison with Homomorphic Encryption

Paillier encryption provides exact results with IND-CPA security and requires no shuffler. Each client encrypts its bit count s_t under a common public key (one ciphertext of 2κ bits, e.g., 4096 bits at $\kappa = 128$), the server multiplies the k ciphertexts ($O(k)$ modular multiplications), and a designated decryption party recovers the sum. The total communication is $O(k\kappa)$ bits for the ciphertexts plus $O(\kappa)$ for the result.

PolyVeil's compressed variant requires $O(k)$ scalars of 64 bits each, giving $O(64k)$ bits total. For Paillier, the cost is $O(4096k)$ bits, a factor of 64 larger. This bandwidth gap may or may not be significant depending on the deployment, but it illustrates the lightweight nature of PolyVeil's communication.

The more important distinction concerns infrastructure requirements. Paillier encryption requires a *public-key infrastructure* (PKI), meaning that someone must generate the key pair (pk, sk), distribute pk to all clients, and safeguard sk. If the decryption key sk is held by a single party and that party is compromised (by an attacker, a subpoena, or an insider), all past and future encrypted values can be decrypted, retrospectively destroying the privacy of every client. Threshold variants distribute sk among multiple parties to mitigate this risk, but they add significant protocol complexity (distributed key generation, threshold decryption rounds, handling of corrupted shareholders).

PolyVeil requires no PKI at all. There are no encryption keys, no decryption keys, no key distribution ceremonies, and no key storage. The only trust infrastructure is the shuffler, which processes ephemeral scalar values and need not maintain any long-term secrets. If the shuffler is compromised after the protocol completes, no retroactive damage occurs (the shuffler does not hold any key that could decrypt past sessions). This absence of long-term secrets is a significant operational advantage in deployment scenarios where key management is expensive, error-prone, or politically contentious (e.g., multi-jurisdictional health data aggregation where no single entity is trusted to hold a decryption key).

5.4 Communication Complexity

Theorem 5.1. *The total communication is $O(kn^2)$ for the full protocol and $O(k)$ for the compressed variant. The shuffle requires $O(k)$ communication.*

Proof. In the full protocol, each client sends a $2n \times 2n$ matrix ($O(n^2)$ entries) and the shuffler handles k scalars. In the compressed variant, each client sends one scalar and the shuffler handles k scalars. □

5.5 Attack Resilience

5.5.1 The Carlini et al. Attack on InstaHide and Why It Does Not Apply

InstaHide [10] was broken by Carlini et al. [11]. We describe their attack precisely and explain why each step fails against PolyVeil.

In InstaHide, a training dataset of private images $\{x_1, \ldots, x_m\}$ is encoded for release. Each encoding $\tilde{x}_a$ combines a private image with randomly selected public images via $\tilde{x}_a = \sum_{i \in T_a} c_{a,i}\, \sigma_{a,i} \odot x_i$, where T_a is a random subset of images, $c_{a,i}$ are mixing coefficients, $\sigma_{a,i} \in \{+1, -1\}^d$ are random coordinate-wise sign vectors, and $\odot$ denotes the Hadamard (element-wise) product. A critical feature is that each private image appears in *multiple* encodings across the dataset.

The Carlini et al. attack has two stages. In the first stage (graph recovery), the attackers compute the coordinate-wise squared inner product $\sum_{d'} (\tilde{x}_a)^2_{d'} (\tilde{x}_b)^2_{d'}$ between all pairs (a, b) of encodings. The random signs σ are eliminated by the squaring, and when two encodings a and b share a common private image x_i (i.e., $i \in T_a \cap T_b$), this quantity is detectably larger than when they share no private image. This allows the attackers to reconstruct a bipartite graph between encodings and private images. In the second stage (image recovery), the graph structure determines which encodings correspond to which private images, and the mixing equations $\tilde{x}_a = \sum_i c_{a,i} \sigma_{a,i} \odot x_i$ become a solvable linear system.

This attack fails against PolyVeil for two specific structural reasons. First, PolyVeil generates exactly one masked matrix D_t per client per protocol execution. There is no dataset of multiple encodings sharing common private components, so the graph recovery stage has no correlations to detect. Even across independent protocol executions (with fresh random decoy permutations each time), the masked matrices are statistically independent, providing no cross-execution signal. Second, PolyVeil uses convex matrix combinations rather than element-wise sign flips. The Hadamard product structure of InstaHide is what makes the squared-inner-product trick work (squaring eliminates the random signs coordinate by coordinate). PolyVeil's convex combination of permutation matrices has no analogous coordinate-wise structure; the entries of D_t are sums of contributions from multiple permutations, and there is no operation that "cancels" the decoy components while preserving the private component.

5.6 Post-Quantum Security

In the two-layer protocol, the server's security is information-theoretic and therefore immune to quantum attack. The aggregator's security against likelihood-based attacks involves the #P-hardness of mixed discriminants and permanent-related computations. No quantum algorithm is known to compute the permanent in polynomial time, and the problem is widely believed to be hard for quantum computers (it is #P-hard, and #P is believed to be outside BQP). However, the quantum hardness of the permanent is not proved, and full aggregator security (Conjecture 4.5) depends on open complexity-theoretic questions. The post-quantum status of PolyVeil's computational layer is therefore plausible but not certain.

5.7 Relationship to Ciphertext Indistinguishability

Standard ciphertext indistinguishability (IND-CPA) requires that no polynomial-time adversary can distinguish encryptions of two chosen plaintexts with non-negligible advantage. In the two-layer protocol, the server achieves a guarantee strictly stronger than IND-CPA: the server's view (F, H) is *identically distributed* (statistical distance zero) for any two inputs with the same aggregate, against

all adversaries regardless of computational power. This is the information-theoretic analogue of IND-CPA, sometimes called perfect secrecy for the server entity.

The aggregator faces a weaker (computational) guarantee: it can compute the posterior distribution over M_t given D_t in principle, but the natural approach (likelihood evaluation) is #P-hard (Theorem 4.6). Whether all polynomial-time attacks are ruled out remains an open conjecture (Conjecture 4.5).

5.8 Potential Attack Vectors

Collusion. If c clients reveal their η_t values to the server outside the shuffle, the server can compute $s_t = (f_t - \eta_t)/\alpha^*$ for those clients, reducing the anonymity set from k to $k - c$.

Malicious server. A server that deviates from the protocol (e.g., using different extraction vectors for different clients) could extract additional linear functionals of D_t. Extending PolyVeil to the malicious model would require verifiable computation.

Repeated executions with correlated data. Each execution is independently secure with fresh randomness. However, if the same clients run the protocol on correlated bitstreams across rounds, the server may detect correlations in f_t values. Cross-round correlation attacks remain an open question.

Small k. The shuffle provides $1/k$ anonymity per client. For $k = 3$, this is weak. Strong privacy requires large k.

5.9 Comparison with Alternative Approaches

Approach	Server security	Aggregator security	PKI
Trusted aggregator	IT (trivial)	None	No
Bonawitz et al. [4]	IT	IT (pairwise masks)	$O(k^2)$ setup
Paillier HE [5]	IND-CPA	N/A (single server)	Yes
Two-Layer PolyVeil	IT (Thm. 4.2)	Likelihood attacks #P-hard (Thm. 4.6)	No

Two-Layer PolyVeil occupies a unique point: it achieves information-theoretic server security without PKI, and computational aggregator security from combinatorial (#P) hardness rather than number-theoretic hardness. The trade-off is the non-collusion assumption between server and aggregator, which is also required by trusted-aggregator and two-server schemes.

The Bonawitz et al. scheme achieves IT security for both entities but requires $O(k^2)$ pairwise key agreement. PolyVeil requires only $O(k)$ communication (each client sends one $O(n^2)$-entry matrix to the aggregator and one scalar to the noise aggregator) and no key agreement.

6 Conclusion

We have presented PolyVeil and introduced Combinatorial Privacy as a paradigm for privacy-preserving aggregation.

Our analysis proceeded in three stages. First, we described two weak protocols (full matrix and compressed) that are vulnerable to a de-shuffling attack. Second, we identified the fatal vulnerability: the server can recover all individual data with probability 1 by exploiting the integrality constraint on bit counts (Theorem 3.2). Third, we developed the Two-Layer PolyVeil protocol (Algorithm 3), which achieves provable security through a separation-of-information architecture. The main server receives only aggregate scalars and achieves *perfect simulation-based security* (Theorem 4.2): its view is identically distributed for any two inputs with the same aggregate, against all adversaries regardless of computational power. A separate aggregator receives Birkhoff-encoded matrices but faces a computational barrier: we proved that likelihood-based inference requires solving #P-hard problems (Theorem 4.6), specifically the mixed discriminant and permanent. Whether all polynomial-time attacks (not just likelihood-based ones) are ruled out remains an open conjecture (Conjecture 4.5).

The Birkhoff polytope plays no role in the server's information-theoretic security (any aggregation protocol achieves that), but it is *essential* for the aggregator's computational barrier. Replacing the Birkhoff encoding with Gaussian noise would make the aggregator's inference trivially easy (Remark 4.7). This specificity — computational hardness from the combinatorial structure of a polytope's decompositions — is what distinguishes Combinatorial Privacy from both noise-based (DP) and number-theoretic (MPC, HE) approaches.

Future directions include formalizing average-case hardness of the aggregator's inference problem, extending to the malicious model, and investigating whether non-likelihood attacks can also be ruled out.

A Worked Example (Weak Protocols)

We trace every computation explicitly for $k = 3$ clients, $n = 2$ bits, $\alpha^* = 0.3$ to illustrate the deshuffling attack. The example matches the weak full protocol (Algorithm 1) but the same attack works on the compressed variant.

A.1 Ground Truth

Client 1 holds $\mathbf{b}_1 = (1, 0)$, so $s_1 = 1$. Client 2 holds $\mathbf{b}_2 = (1, 1)$, so $s_2 = 2$. Client 3 holds $\mathbf{b}_3 = (0, 1)$, so $s_3 = 1$. The true aggregate is $S = 1 + 2 + 1 = 4$.

The public parameters are $n = 2$, $\alpha^* = 0.3$, $\mathbf{w} = (1, 0, 1, 0)^T$, $\mathbf{y} = (0, 1, 0, 1)^T$.

A.2 Client-Side Masking (Full Detail for Client 1)

Encoding M_1. Client 1's bitstream is $(1, 0)$. Applying Definition 2.6:

$$M_1 = \text{blockdiag}(\Pi(1),\ \Pi(0)) = \text{blockdiag}\left(\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix},\ \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}\right) = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Generating decoy permutations. Client 1 chooses $K_1 = 2$ decoy permutations drawn uniformly from S_4 (the symmetric group on $\{1, 2, 3, 4\}$). Suppose the Fisher–Yates shuffle produces:

Permutation $\sigma_1 = (3, 1, 4, 2)$, meaning $\sigma_1(1) = 3$, $\sigma_1(2) = 1$, $\sigma_1(3) = 4$, $\sigma_1(4) = 2$. The corresponding permutation matrix has $P_{1,1}[i, \sigma_1(i)] = 1$:

$$P_{1,1} = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix}.$$

Permutation $\sigma_2 = (2, 1, 4, 3)$:

$$P_{1,2} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

Choosing coefficients. Client 1 samples $\alpha_{1,1} = 0.5$ and $\alpha_{1,2} = 0.2$, satisfying $\alpha_{1,1} + \alpha_{1,2} = 0.7 = 1 - 0.3 = 1 - \alpha^*$.

Constructing D_1. $D_1 = 0.3M_1 + 0.5P_{1,1} + 0.2P_{1,2}$:

$$D_1 = \begin{pmatrix} 0 & 0.5 & 0.5 & 0 \\ 1.0 & 0 & 0 & 0 \\ 0 & 0 & 0.3 & 0.7 \\ 0 & 0.5 & 0.2 & 0.3 \end{pmatrix}.$$

Computing η_1. $\eta_1 = 0.5 \times (\mathbf{w}^T P_{1,1}\mathbf{y}) + 0.2 \times (\mathbf{w}^T P_{1,2}\mathbf{y}) = 0.5 \times 1 + 0.2 \times 2 = 0.9$.

Client 1 sends D_1 to the server and holds $\eta_1 = 0.9$ for the shuffle.

A.3 Client-Side Masking for Clients 2 and 3 (Summary)

Client 2. $\mathbf{b}_2 = (1, 1)$, $s_2 = 2$, $\eta_2 = 0.8$, $f_2 = \alpha^* s_2 + \eta_2 = 1.4$.

Client 3. $\mathbf{b}_3 = (0, 1)$, $s_3 = 1$, $\eta_3 = 0.35$, $f_3 = \alpha^* s_3 + \eta_3 = 0.65$.

A.4 Server-Side Extraction and De-Shuffling

The server receives $f_1 = 1.2$, $f_2 = 1.4$, $f_3 = 0.65$ (linked) and shuffled η values $\{0.8, 0.35, 0.9\}$. The server tests all $3! = 6$ assignments of the η values to clients, computing $\hat{s}_t = (f_t - \eta_{\sigma(t)})/\alpha^*$ and checking whether all $\hat{s}_t$ are integers in $\{0, 1, 2\}$. Only the true assignment yields valid bit counts $(1, 2, 1)$. Hence the server recovers all s_t with probability 1.

A Background on Simulation-Based Security Proofs

This appendix provides a self-contained introduction to the simulation paradigm for proving security of cryptographic protocols. It is a prerequisite for understanding the security proofs in Section 4.

A.1 The Problem That Simulation Solves

Consider a protocol Π where k parties hold private inputs $x_1, \ldots, x_k$ and jointly compute a function $f(x_1, \ldots, x_k) = y$. During execution, each party sends and receives messages. The *view* of party i, denoted $\mathrm{View}_i^{\Pi}(x_1, \ldots, x_k)$, is the random variable consisting of party i's input x_i, its random coins r_i, and the sequence of all messages $m_1, m_2, \ldots$ it receives during execution. The fundamental question is whether View_i reveals information about other parties' inputs beyond what is already implied by x_i and y.

Intuitively, a protocol is "secure" if View_i is "no more informative" than (x_i, y). The simulation paradigm formalizes this by requiring the existence of an algorithm that can *fabricate* a fake view, using only (x_i, y) as input, such that the fake view is distributed identically to the real view.

A.2 Formal Definition

Definition A.1 (Simulation-based security, semi-honest model)**.** *A k-party protocol* Π securely computes f *in the semi-honest model if for each party $i \in [k]$, there exists a probabilistic polynomial-time algorithm $\mathcal{S}_i$ (the* simulator *for party i) such that for all input vectors $(x_1, \ldots, x_k)$ in the domain of f:*

$$\{\mathcal{S}_i(x_i,\ f(x_1, \ldots, x_k))\} \ \equiv\ \left\{\mathrm{View}_i^{\Pi}(x_1, \ldots, x_k)\right\},$$

where $\equiv$ denotes either identical distributions (information-theoretic, or perfect *security) or computational indistinguishability (computational security). The left-hand side is the simulator's output distribution; the right-hand side is the real view's distribution.*

The simulator $\mathcal{S}_i$ receives *only* what party i is "supposed to know" after the protocol ends: its own input x_i and the output $y = f(x_1, \ldots, x_k)$. It does *not* receive any other party's input, the random coins of other parties, or the messages exchanged during the real protocol. Despite this limited input, the simulator must produce a fake transcript whose distribution matches the real one perfectly (or computationally indistinguishably).

A.3 How to Construct a Simulation Proof

A simulation proof proceeds in three stages.

In the first stage, one precisely defines the real view. For PolyVeil, the server's real view consists of the doubly stochastic matrices $D_1, \ldots, D_k$ (each linked to a client identity) and the shuffled noise sequence $(\eta_{\pi(1)}, \ldots, \eta_{\pi(k)})$. These are random variables whose joint distribution depends on all clients' inputs $\mathbf{b}_1, \ldots, \mathbf{b}_k$ and the random coins (decoy permutations, coefficients, shuffle permutation).

In the second stage, one constructs the simulator $\mathcal{S}$. The simulator receives only the aggregate $S = \sum_t s_t$ and the public parameters (k, n, α^*). It must produce a fake tuple $(D'_1, \ldots, D'_k, \tilde{\eta}'_1, \ldots, \tilde{\eta}'_k)$ with the same distribution as the real view. The typical construction is: choose fictitious inputs $\mathbf{b}'_t$ with $\sum s'_t = S$, run the real protocol honestly with these fictitious inputs and fresh randomness, and output the resulting messages.

In the third stage, one proves that the simulator's output is distributed identically to the real view. This is usually the hardest step. It requires showing that the joint distribution of all messages is the same regardless of which specific inputs (with the same aggregate S) were used. The proof typically identifies which components of the view depend on the private inputs and which do not, and shows that the input-dependent components are "masked" by the input-independent randomness.

A.4 Why Simulation Implies Security

Proposition A.2. *If a simulator $\mathcal{S}_i$ with $\mathcal{S}_i(x_i, y) \equiv \mathrm{View}_i^{\Pi}(x_1, \ldots, x_k)$ exists, then for any function g (representing any "information extraction" strategy):*

$$g\big(\mathrm{View}_i^{\Pi}(x_1, \ldots, x_k)\big) \;\equiv\; g(\mathcal{S}_i(x_i, y))\,.$$

Proof. If two random variables X and Y are identically distributed ($X \equiv Y$), then for any measurable function g, $g(X) \equiv g(Y)$. This is a basic property of pushforward measures. Applying it with $X = \mathrm{View}_i^{\Pi}$ and $Y = \mathcal{S}_i(x_i, y)$ gives the result. □

The consequence is that any information $g(\mathrm{View}_i)$ that the adversary extracts from the real view could equally well have been extracted from (x_i, y) alone (via $g(\mathcal{S}_i(x_i, y))$, which is a function only of x_i and y). Therefore the protocol reveals no information beyond what (x_i, y) already implies.

A.5 Application to PolyVeil

In PolyVeil, the "adversary" is the honest-but-curious server. The server has no private input of its own ($x_i = \emptyset$). The function being computed is $f(\mathbf{b}_1, \ldots, \mathbf{b}_k) = S = \sum_t s_t$. Therefore the simulator receives only (S, k, n, α^*).

In the weak protocols, the server sees identity-linked values that allow deterministic de-shuffling (Theorem 3.2), so the basic protocol does not achieve simulation-based security. The Two-Layer PolyVeil protocol corrects this: the server receives only two aggregate scalars (F, H), and its view is perfectly simulable from the aggregate S alone (Theorem 4.2). The aggregator, which sees individual Birkhoff-encoded matrices D_t but not the noise values η_t, faces a computational barrier: recovering M_t from D_t requires evaluating a density that is #P-hard to compute (Theorem 4.6).

References

[1] A. C. Yao, "How to generate and exchange secrets," in *FOCS*, IEEE, 1986, pp. 162–167.

[2] O. Goldreich, S. Micali, and A. Wigderson, "How to play any mental game," in *STOC*, ACM, 1987, pp. 218–229.

[3] I. Damgård, V. Pastro, N. Smart, and S. Zakarias, "Multiparty computation from somewhat homomorphic encryption," in *CRYPTO*, Springer, 2012, pp. 643–662.

[4] K. Bonawitz et al., "Practical secure aggregation for privacy-preserving machine learning," in *CCS*, ACM, 2017, pp. 1175–1191.

[5] P. Paillier, "Public-key cryptosystems based on composite degree residuosity classes," in *EUROCRYPT*, Springer, 1999, pp. 223–238.

[6] C. Gentry, "Fully homomorphic encryption using ideal lattices," in *STOC*, ACM, 2009, pp. 169–178.

[7] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *TCC*, Springer, 2006, pp. 265–284.

[8] A. Cheu, A. Smith, J. Ullman, D. Zeber, and M. Zhilyaev, "Distributed differential privacy via shuffling," in *EUROCRYPT*, Springer, 2019, pp. 375–403.

[9] B. Balle, J. Bell, A. Gascón, and K. Nissim, "The privacy blanket of the shuffle model," in *CRYPTO*, Springer, 2019, pp. 638–667.

[10] Y. Huang, Z. Song, K. Li, and S. Arora, "InstaHide: Instance-hiding schemes for private distributed learning," in *ICML*, 2020.

[11] N. Carlini, S. Chien, M. Nasr, S. Song, A. Terzis, and F. Tramer, "Is private learning possible with instance encoding?," in *IEEE S&P*, 2021.

[12] A. Shamir, "How to share a secret," *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.

[13] M. Chase, E. Ghosh, and O. Poburinnaya, "Secret-shared shuffle," in *ASIACRYPT*, Springer, 2020, pp. 342–372.

[14] L. G. Valiant, "The complexity of computing the permanent," *Theoretical Computer Science*, vol. 8, no. 2, pp. 189–201, 1979.

[15] A. Barvinok, "Two algorithmic results for the traveling salesman problem," *Mathematics of Operations Research*, vol. 21, no. 1, pp. 65–84, 1996.

[16] R. J. Lipton, "New directions in testing," in *Distributed Computing and Cryptography*, DIMACS Series, vol. 2, AMS, 1991, pp. 191–202.

[17] R. A. Brualdi, "Notes on the Birkhoff algorithm for doubly stochastic matrices," *Canadian Mathematical Bulletin*, vol. 25, no. 2, pp. 191–199, 1982.

[18] M. Marcus and R. Ree, "Diagonals of doubly stochastic matrices," *The Quarterly Journal of Mathematics*, vol. 10, no. 1, pp. 296–302, 1959.

[19] F. Dufossé and B. Uçar, "Notes on Birkhoff–von Neumann decomposition of doubly stochastic matrices," *Linear Algebra and its Applications*, vol. 497, pp. 108–115, 2016.

[20] G. M. Ziegler, *Lectures on Polytopes*, vol. 152, Springer, 2012.

[21] A. Schrijver, *Combinatorial Optimization: Polyhedra and Efficiency*, vol. 24, Springer, 2003.